

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development and Jira

Practical – V (Self Study)

Roll No.: T016	Name: Himani Malankar
Class:TYIT	Batch:1
Date of Assignment: 29/07/2026	Date/Time of Submission: 29/07/2026

A. Design PEAS description for an Online Food Delivery Agent and classify its environment.

CODE:

```
table = {
    ('Yes', 'Yes', 'No'): 'Accept Order',
    ('Yes', 'No', 'No'): 'Wait for Restaurant',
    ('Yes', 'Yes', 'Yes'): 'Deliver Order',
    ('No', 'Yes', 'No'): 'Wait for Order',
    ('No', 'No', 'No'): 'Stay Idle',
    ('No', 'Yes', 'Yes'): 'Return to Restaurant',
    ('Yes', 'No', 'Yes'): 'Pick Up Order',
    ('No', 'No', 'Yes'): 'Go Offline'
}

percepts = []

def food_delivery_agent(percept):
    percepts.append(percept)
    action = table.get(percept, "Wait")
    return action

print("Online Food Delivery Table-Driven Agent")

n = int(input("Enter number of percepts: "))

for i in range(n):
    print(f"\nPercept {i + 1}")

    order = input("Order Available? (Yes/No): ").capitalize()
    restaurant = input("Restaurant Ready? (Yes/No): ").capitalize()
    delivery = input("Delivery Assigned? (Yes/No): ").capitalize()

    percept = (order, restaurant, delivery)
    action = food_delivery_agent(percept)

    print("Current Percept:", percept)
    print("Agent Action:", action)

print("Himani T016")
```

OUTPUT:

```
''' Online Food Delivery Table-Driven Agent
```

```
Percept 1
```

```
Current Percept: ('Yes', 'Yes', 'No')
```

```
Agent Action: Accept Order
```

```
Percept 2
```

```
Current Percept: ('No', 'No', 'No')
```

```
Agent Action: Stay Idle
```

```
Percept 3
```

```
Current Percept: ('Yes', 'Yes', 'Yes')
```

```
Agent Action: Deliver Order
```

```
Percept 4
```

```
Current Percept: ('No', 'Yes', 'Yes')
```

```
Agent Action: Return to Restaurant
```

```
Himani T016
```

B. Implement Breadth-First Search to find the shortest path in a simple maze.

CODE:

```
from collections import deque

maze = [
    ['S', '.', '.', '#'],
    ['#', '.', '#', '.'],
    ['.', '.', '.', '.'],
    ['#', '#', '.', 'G']
]

rows, cols = len(maze), len(maze[0])

# Find Start and Goal positions
for i in range(rows):
    for j in range(cols):
        if maze[i][j] == 'S':
            start = (i, j)
        if maze[i][j] == 'G':
            goal = (i, j)

queue = deque([(start, [start])])
visited = {start}

while queue:
    (x, y), path = queue.popleft()

    if (x, y) == goal:
        print("Shortest Path:", path)
        break

    for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nx, ny = x + dx, y + dy

        if (
            0 <= nx < rows and
            0 <= ny < cols and
            maze[nx][ny] != '#' and
            (nx, ny) not in visited
        ):
            visited.add((nx, ny))
            queue.append(((nx, ny), path + [(nx, ny)]))

print("Himani T016")
```

OUTPUT:

```
... Shortest Path: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (3, 2), (3, 3)]

Himani T016
```

C. Apply Bayes' Rule to calculate the probability of rain given cloudy weather.

CODE:

```
P_rain = 0.3
P_cloudy_given_rain = 0.8
P_cloudy = 0.5

# Bayes' Theorem
P_rain_given_cloudy = (P_cloudy_given_rain * P_rain) / P_cloudy

print("Probability of Rain given Cloudy Weather =",
      round(P_rain_given_cloudy, 2))

print("Himani T016")
```

OUTPUT:

```
... Probability of Rain given Cloudy Weather = 0.48
Himani T016
```

D. Solve the Missionaries and Cannibals problem using BFS or DFS.

CODE:

```
from collections import deque

start = (3, 3, 1)
goal = (0, 0, 0)

def valid(s):
    m, c, _ = s
    return (m == 0 or m >= c) and (3 - m == 0 or 3 - m >= 3 - c)

moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

queue = deque([(start, [start])])
visited = {start}

while queue:
    (m, c, b), path = queue.popleft()

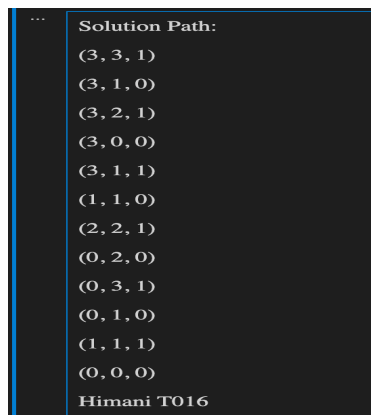
    if (m, c, b) == goal:
        print("Solution Path:")
        for p in path:
            print(p)
        break

    for dm, dc in moves:
        if b:
            ns = (m - dm, c - dc, 0)
        else:
            ns = (m + dm, c + dc, 1)

        if (
            0 <= ns[0] <= 3 and
            0 <= ns[1] <= 3 and
            valid(ns) and
            ns not in visited
        ):
            visited.add(ns)
            queue.append((ns, path + [ns]))

print("Himani T016")
```

OUTPUT:



```
... Solution Path:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
Himani T016
```

E. Implement A* Search for a graph and compare its result with UCS.

CODE:

```
import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 4)],
    'E': [('G', 1)],
    'F': [('G', 2)],
    'G': []
}

h = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 4,
    'E': 2,
    'F': 1,
    'G': 0
}

def search(start, goal, astar=True):
    pq = [(0, start, [start])]
    visited = set()

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
            return path, cost

        if node in visited:
            continue

        visited.add(node)

        for nxt, w in graph[node]:
            g = cost - h[node] if astar else cost
            ncost = g + w + (h[nxt] if astar else 0)
            heapq.heappush(pq, (ncost, nxt, path + [nxt]))

    ucs_path, ucs_cost = search('A', 'G', False)
    astar_path, astar_cost = search('A', 'G', True)

    print("UCS Path :", ucs_path, "Cost :", ucs_cost)
    print("A* Path :", astar_path, "Cost :", astar_cost)

    print("Himani T016")
```

OUTPUT:

```
... UCS Path : ['A', 'B', 'D', 'G'] Cost : 7  
A* Path : ['A', 'B', 'D', 'G'] Cost : 0  
Himani T016
```